Classes of computers

1. Desktop computers:
   - General purpose
   - Subject to cost/performance tradeoff
2. Server computers:
   - Network based
   - High capacity, performance, reliability
   - Range from small servers to building sized
3. Embedded computers
   - Hidden as component of systems
   - Stigent power/performance/cost constraints

Understanding performance

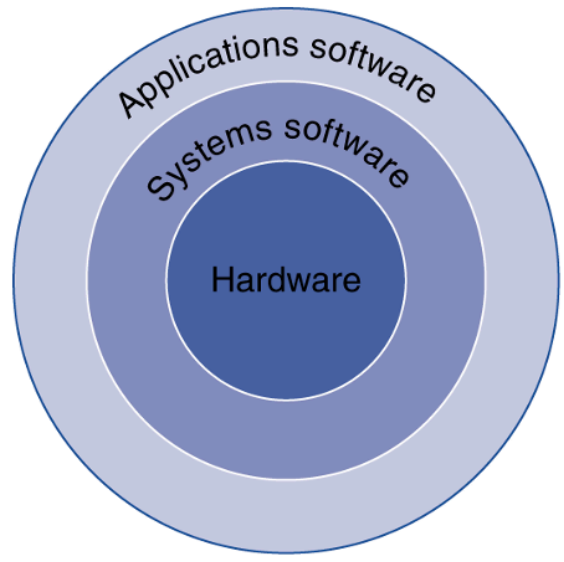
1. Algorithm:
   - Determines the no. of operations executed
2. Programming language, compiles, architecture:
   - Determine no. of machine instructions executed per operation
3. Processor & memory system
   - Determine how fast instructions are executed
4. I/O system (including OS)
   - Determines how fast I/O operations are executed

- This is essentially telling us components that can "impact" performance.
- Each layer's performance is also dependent on the one before it.

Below Your Program

- Application Software
  - Written in high-level language

- System software
  - Compiler: translates HLL code to machine code
  - Operating system: service the code:
    - Handling I/O
    - Managing memory & storage
    - Scheduling tasks & sharing resources
- Hardware:
  - Processor, memory, I/O controller

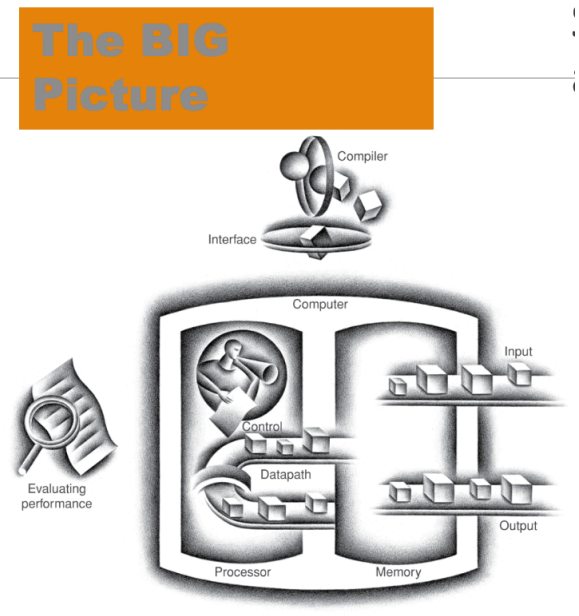


Levels of Program code:

- High level language:
  - Level of abstraction closer to problem domain
  - Provides for productivity & portability
- Assembly language:
  - Textual representation of instruction
- Hardware representation
  - Binary digits (bits)
  - Encoded instructions & data

Components of a computer:

- Same components for all kinds of computer
  - Desktop, server, embedded
- I/O includes
  1. User-interface device
     - Display, keyboard, mouse
  2. Storage devices
     - Harddisk, CD/DVD, flash
  3. Network adapters
     - For communicating with other computers



Inside the CPU (Processor)

- Datapath: performs operations on data
- Control: sequences datapath, memory...
- Cache memory:
  - Small fast SRAM memory for immediate access to data.

A safe place for data:

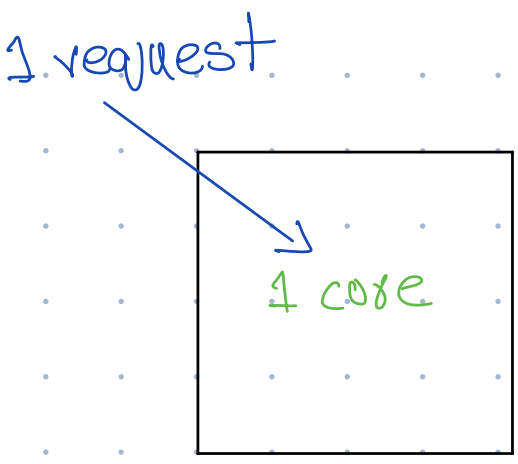
1. Volatile memory
   - Loses instructions & data when power off
2. Non-volatile secondary memory
   - Magnetic disk
   - Flash memory
   - Optical disk

Networks:

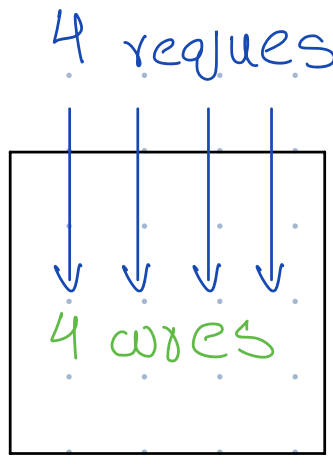
- Communication & resource sharing
- Local Area Network (LAN):
  - Ethernet
  - Within a building
- Wide Area Network (WAN):
  - the internet
- Wireless network:
  - Wifi
  - Bluetooth

Response time & throughput:

- Response time:
  How long it takes to do a task.
- Throughput:
  Total work done per unit time



↳ Takes 4 seconds



↳ still takes 4 seconds but throughput is more.

↳ How are response time & throughput affected by:

Replacing the processor with a faster version?

↳ Same response time (unless parallelism helps (see moore's law for why)

Adding more processors?

↳ More throughput

Multi-processors

↳ Multi-core microprocessors

↳ More than one processor per chip

↳ Requires explicit parallel programming

↗ doesn't require multi-processor

- Compare with instruction level parallelism
  - Hardware executes multiple instructions at once.
  - Hidden from programmer
- Hard to do
  - Programming for performance
  - Load balancing
  - Optimizing communication & synchorization

Moore's Law:

No. of transistors on an affordable CPU would double every 18 months.

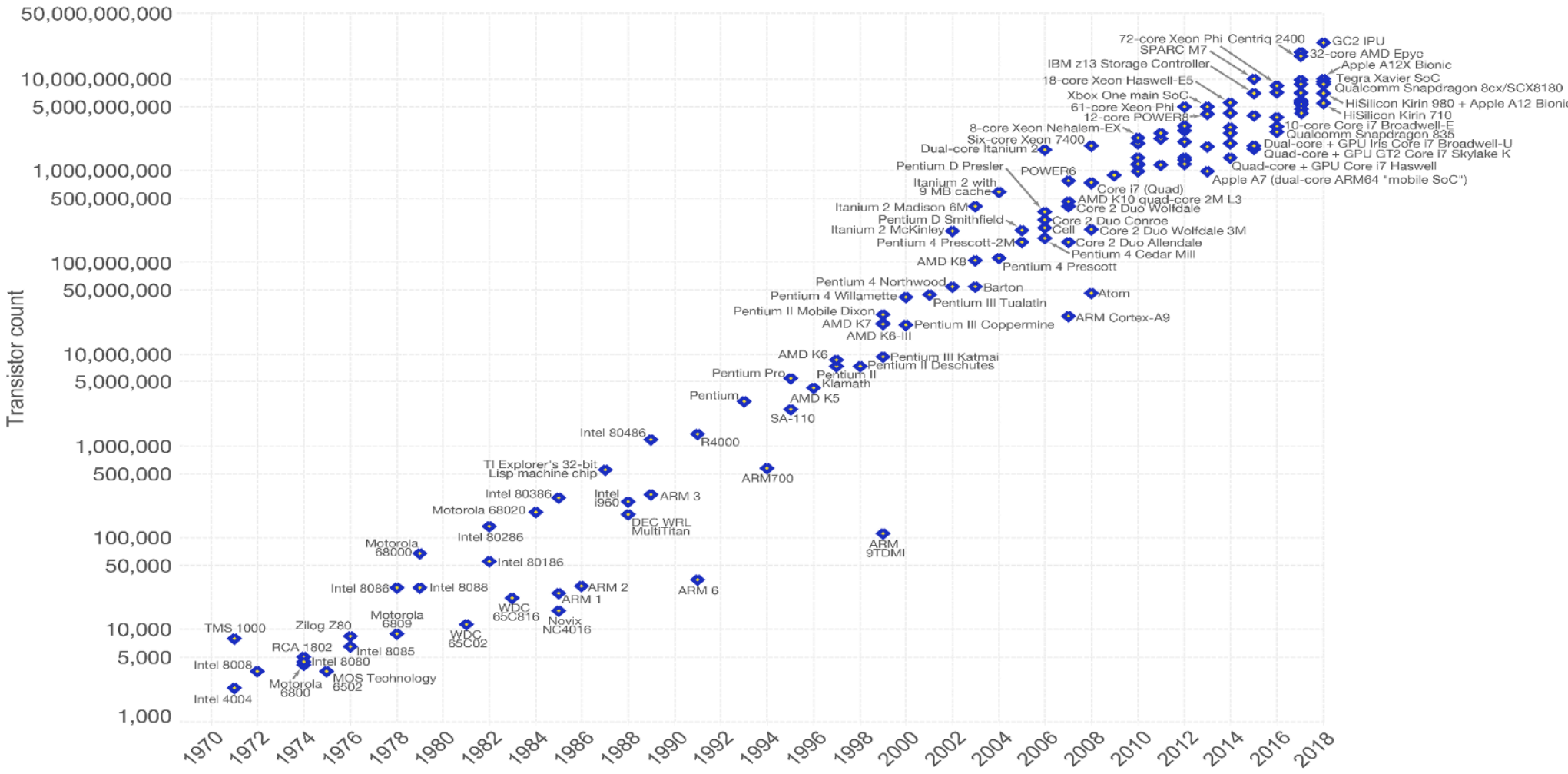
OR

"Processing speed, or overall processing power will double every 18 months"

Moore's Law – The number of transistors on integrated circuit chips (1971-2018)

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important as other aspects of technological progress – such as processing speed or the price of electronic products – are linked to Moore's law.

Our World in Data



↳ In the 1970s, No. of transistors doubled ✓ so did processor speed.

↳ Here both interpretation of Moore's law holds

↳ In 2000 to 2009, no. of transistors continued to double but processor speed slowed down.

↳ Here, the other interpretation is more accurate

Why?

↳ The increase transistors was due to multi-core processor but processor speed remained relatively same.

↳ Hence, 2000-2009 effect

↳ This was due to bottle-neck due to inter-process interaction (process on core 1 may need to talk to process on core 2)

Why we needed multi-core systems?

↳ Adding more transistors increased heat dispation

↳ To add more transistors, we need to make them small but that led to Quantum leakage:

↳ Due to small size, electrons leaked into other transistors

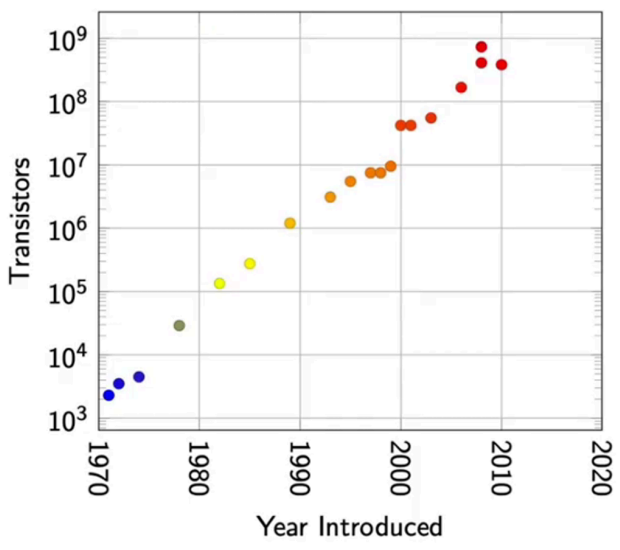
Ultra large-scale integration (ULSI) is the process of embedding millions of transistors on a single silicon semiconductor microchip. (companies did this to follow moore's law)

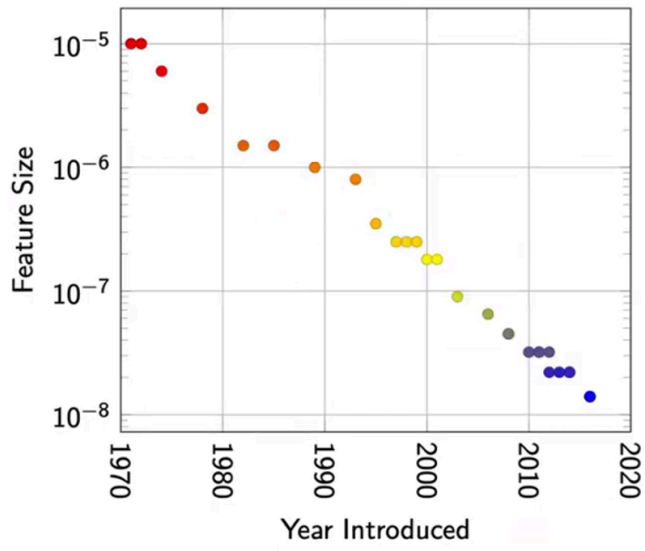
Miniaturization at the atomic level refers to the process of shrinking transistors to sizes approaching the dimensions of individual atoms. This is a critical aspect of advancing semiconductor technology as we strive to fit more transistors onto microchips, enhancing computational power and energy efficiency.

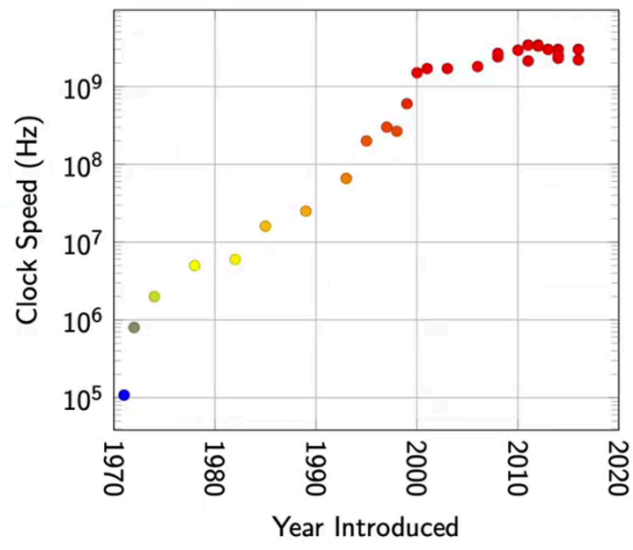
Challenges in Atomic-Level Miniaturization

- **Quantum Effects:**
  - At atomic scales, quantum mechanical phenomena like tunneling become significant. Electrons can "tunnel" through thin barriers, causing leakage currents that waste power and create instability.
- **Material Limitations:**
  - Silicon, the standard material for transistors, faces limitations when structures are shrunk to atomic scales. It struggles with heat dissipation, mechanical strength, and reliability.
- **Manufacturing Precision:**
  - Extreme precision is needed to construct devices at atomic scales. Current lithography techniques, like Extreme Ultraviolet (EUV) lithography, are pushing their limits.
- **Variability:**
  - Manufacturing at atomic scales leads to variations in transistor behavior, which can impact performance consistency.

- **Technologies Enabling Atomic-Level Miniaturization**
  - **FinFET (3D Transistors):**
    - Uses a vertical "fin" structure to increase control over electron flow while maintaining a small footprint.
  - **Gate-All-Around (GAA) Transistors:**
    - Encases the channel with the gate from all sides for better control and reduced leakage.
  - **Carbon Nanotubes:**
    - Strong, conductive, and only a few nanometers in diameter, carbon nanotubes could replace silicon for smaller, faster, and more energy-efficient transistors.
  - **2D Materials:**
    - Materials like graphene and molybdenum disulfide (MoS_2) offer atomic thicknesses and excellent electrical properties.
  - **Quantum Computing:**
    - Exploits quantum mechanics directly instead of shrinking transistors, potentially bypassing traditional limitations.
  - **Molecular Electronics:**
    - Uses individual molecules as electronic components to build devices at the molecular scale.
- **Future Outlook**
  - **Advancements in miniaturization at atomic levels will likely involve:**
    - Transitioning to post-silicon materials.
    - Leveraging quantum physics for computation.
    - Combining top-down and bottom-up fabrication techniques for precision.
    - The miniaturization journey continues to push the boundaries of engineering and physics, with profound implications for computing and technology.





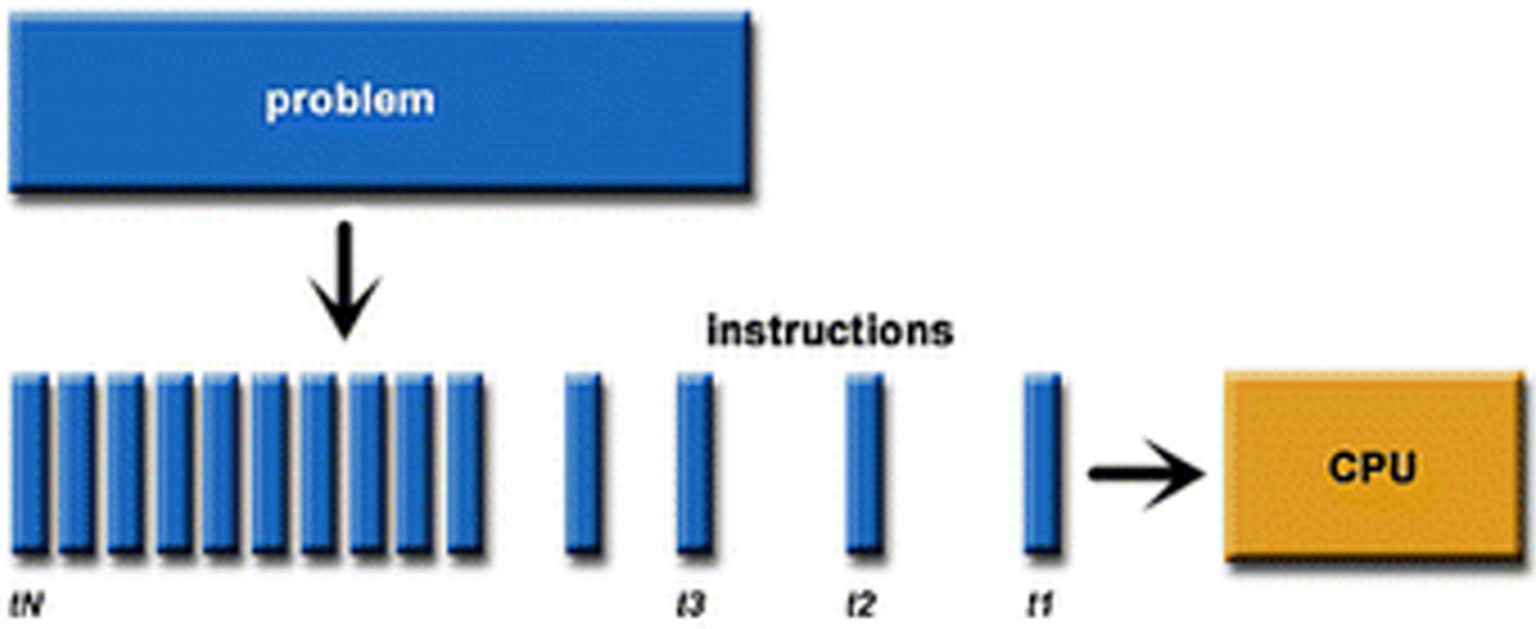


The power wall:

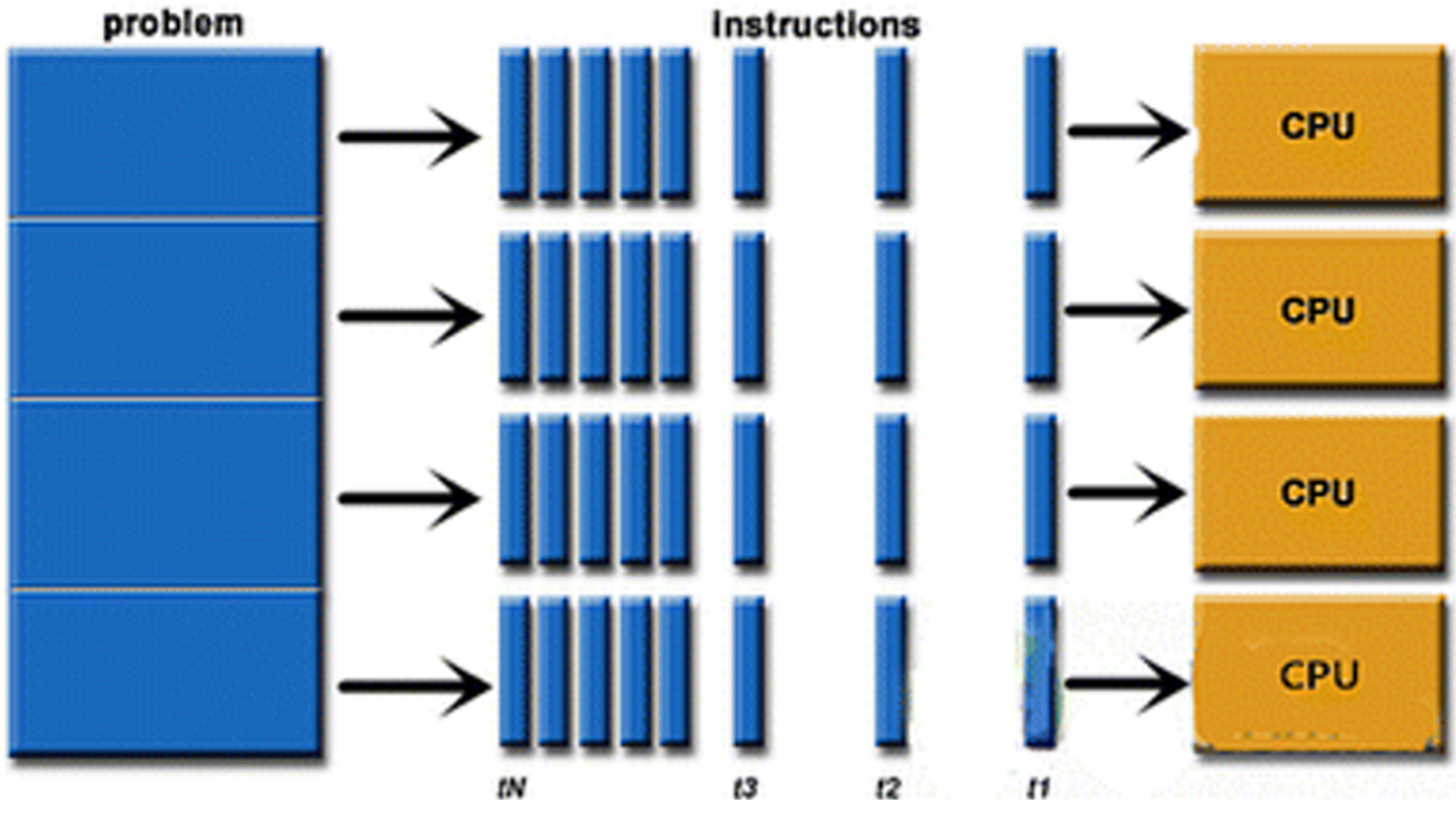
- Heat dissipation increases with number of processors.
- We need to think about something else than increasing number of processors on a single chip.

Parallel Computing Systems

- Serial Variation



- Parallel variation



Classes of Parallelism & Parallel Architectures

Modern computer design relies on parallelism at multiple levels

Introduction:

- "In modern computers, parallelism is not just an idea—it's actually built into the hardware! To make computers faster and more efficient, hardware designers use different methods to execute multiple instructions and tasks at the same time. Let's explore the four major ways this is done."

Energy & cost are key constraints in parallel computing

- **Instruction-Level Parallelism (ILP)**
  - Ask: "Have you ever noticed how a factory assembles products faster when workers handle different steps at the same time?"
  - It also uses speculative execution, where the processor guesses future instructions to speed up processing.
  - **Example:** When a computer loads a webpage, ILP helps process multiple elements at once instead of one by one.
- **Vector Architectures & GPUs**
  - Ask: "How do graphics in video games look so smooth and realistic?"
  - **Example:** When watching a 4K video, your GPU processes thousands of pixels simultaneously instead of one by one.
- **Thread-Level Parallelism (TLP)**
  - Ask: "Have you ever seen a computer with a multi-core processor? What's the advantage?"
  - Example: When playing an online game, one thread renders graphics, another handles game logic, and another manages network communication—all at once!
- **Request-Level Parallelism (RLP)**
- Ask: "Have you ever wondered how cloud services like Google Drive or Netflix handle millions of requests at the same time?"
- **Example:** When you search Google, thousands of servers process parts of the search in parallel and return results almost instantly!

Two major types of parallelism in applications:

Data level Parallelism (DLP):

Example

Applying a formula on data in excel sheet is done simultaneously for all rows

Task level Parallelism (TLP):

Example,

Independent tasks executed in parallel

Your laptop can download a file, play a video, and run a background update all at once because different tasks are handled by different cores in the processor.

Exploiting Parallelism in Hardware

Parallelism is implemented in hardware using four major methods

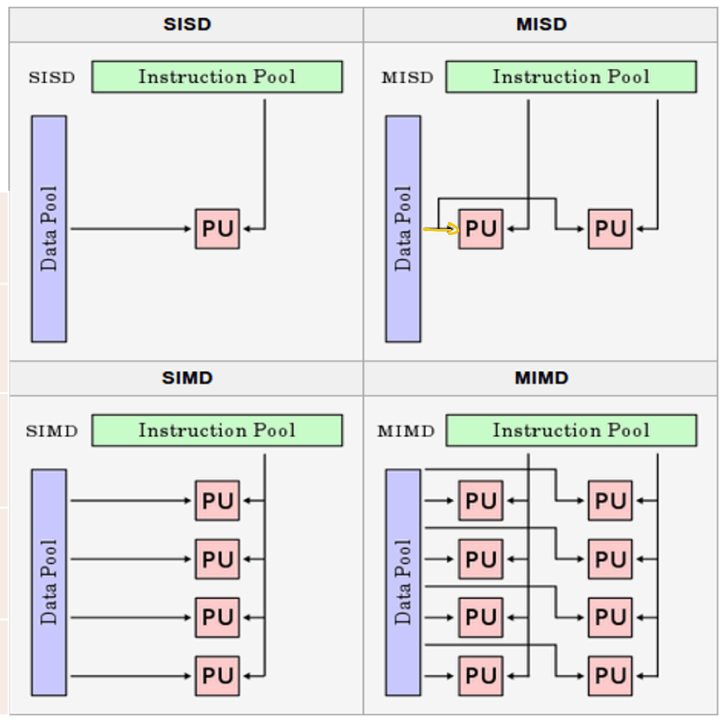
1. **Instruction-Level Parallelism (ILP):**
   1. Uses pipelining and speculative execution.
   2. Exploits modest levels of **DLP**.
2. **Vector Architectures & GPUs:**
   1. Uses a **single instruction** on multiple data (SIMD).
   2. Multimedia instruction sets fall under this category.
3. **Thread-Level Parallelism (TLP):**
   1. Utilizes parallel threads for both **DLP and TLP**.
   2. Requires tightly coupled hardware models.
4. **Request-Level Parallelism (RLP):**
   1. Works on **independent tasks** specified by programmers or the OS.
   2. Used in distributed computing & cloud systems.

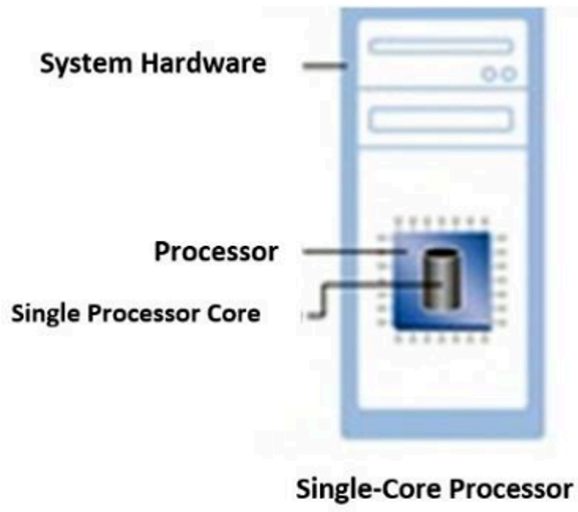
Flynn's Taxonomy

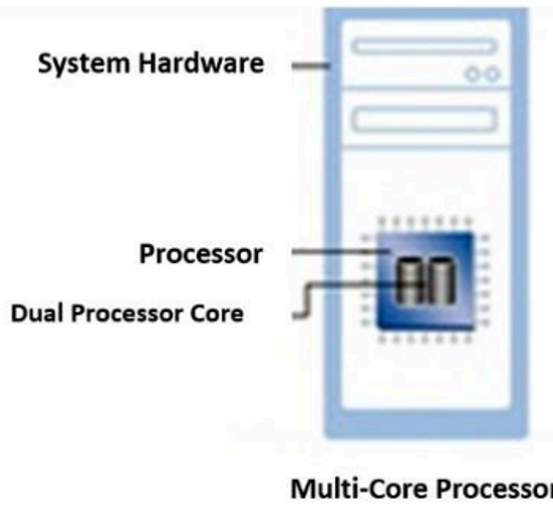
A classification of parallel architectures based on instruction and data streams.

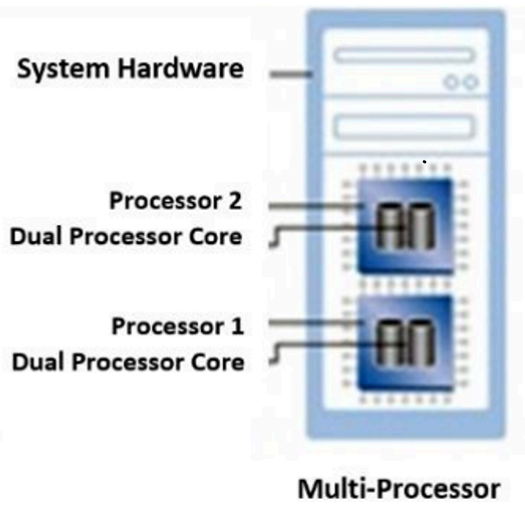
Categorized into four types:

Category	Instruction Stream	Data Stream	Parallelism Type
SISD (Single Instruction, Single Data)	Single	Single	ILP
SIMD (Single Instruction, Multiple Data)	Single	Multiple	DLP
MISD (Multiple Instruction, Single Data)	Multiple	Single	No practical implementation
MIMD (Multiple Instruction, Multiple Data)	Multiple	Multiple	TLP & DLP









Why do we need Parallelism?

1. Solve larger/more complex problems

   ↳ Many problems are so large/complex that it is impractical or impossible to solve them using a serial program, especially given limited computer memory.

2. Provide concurrency:

   ↳ Single compute resource can only do one thing at a time.
   ↳ Multiple compute resources can do many things simultaneously

3. Take advantage of non-local resources

   ↳ Using compute resources on WAN, or even the Internet when local compute resources are scarce or insufficient

4. Much better use of underlying parallel hardware

   ↳ Modern computers are parallel in architecture multi-core/processors

   ↳ Parallel software is specifically intended for parallel hardware with multiple cores, threads etc.

Distributed Systems

Why?

1. Scalable:
   ↳ Ensures that a system can handle increasing workloads by adding more resources as needed

2. Importance of load balancing:

   ↳ Load balancing distributes tasks evenly across

machines to optimize performance & resource utilization

3. Economical
4. Historical
5. functional
   - Many orgs offices are spread over at different locations.

Disadvantages

- keep multiple copies of data
- keeping clocks on different machines synchronized

Supercomputers:

- Defined as one of the most powerful computers
- Just what constitutes "super" varies with time